



CAHIER DES CHARGES

Modèle IA

Détection de fraude · Scoring & NLP africain



PROJET	KWISMO — Protection anti-fraude Mobile Money
DOCUMENT	Cahier des charges — Modèle IA
VERSION	1.0
DATE	21 juillet 2026
DESTINATAIRE	Équipe Data (2 data scientists)
STACK CIBLE	Python · scikit-learn/LightGBM · Hugging Face · FastAPI

Table des matières

1	Présentation du document.....	4
1.1	Deux modèles, un seul service.....	4
1.2	Contrainte fondatrice : léger, rapide, efficace	4
2	Comprendre l'entraînement d'un modèle.....	5
2.1	Le principe : apprendre à partir d'exemples étiquetés.....	5
2.2	Les 7 étapes du cycle de vie.....	5
2.3	Le vocabulaire essentiel.....	5
3	Stratégie de données (adaptée au Cameroun / Afrique)	6
3.1	Sources de données, de la plus fiable à la plus bruitée	6
3.2	Le défi des langues locales	6
3.3	Gouvernance & conformité	6
3.4	Éthique & biais (à surveiller).....	7
4	Modèle A — Scoring de réputation des numéros.....	8
4.1	Caractéristiques (features) utilisées	8
4.2	Algorithme	8
4.3	Entraînement.....	8
5	Règles expertes — protéger dès le premier jour	9
6	Modèle B — Analyse de texte multilingue (NLP)	10
6.1	Prétraitement du texte	10
6.2	Modèle de langue africain	10
6.3	Repli simple garanti.....	10
6.4	Entraînement léger — les techniques clés	10
7	Évaluation & métriques.....	12
7.1	Seuils d'alerte réglables.....	12
8	Apprentissage continu & gestion des versions.....	12
9	Service d'inférence & intégration avec le backend.....	13
9.1	API d'inférence.....	13
9.2	Modes de communication.....	13
9.3	Cohabitation dans le dépôt Git (point critique)	13
10	Architecture & structure des dossiers.....	15
10.1	Choix : Cookiecutter Data Science (adapté)	15

10.2 Arborescence normative.....	15
10.3 Stack technique.....	16
11 Organisation des deux data scientists.....	16
12 Livrables attendus.....	17

1 Présentation du document

Ce cahier des charges décrit le **service de modèle IA de KWISMO**, chargé de détecter les numéros et messages frauduleux liés au Mobile Money. Il est rédigé de façon pédagogique : il explique non seulement *quoi* construire, mais aussi *comment* entraîner les modèles, où trouver les données et *comment* garantir un entraînement léger, rapide et efficace, adapté au contexte camerounais et africain.

1.1 Deux modèles, un seul service

Le service IA contient deux modèles complémentaires, développés en parallèle par les deux data scientists, dans le même dossier :

Modèle	Problème résolu	Nature de la donnée
Modèle A — Scoring de réputation	Évaluer le risque d'un numéro (sécurisé / à signaler / frauduleux).	Comportement : signalements, ancienneté, opérateur, préfixe, cohérence.
Modèle B — Analyse de texte (NLP)	Détecter qu'un message (SMS/WhatsApp) est une arnaque.	Texte en français, anglais, pidgin et franglais camerounais.

Pourquoi deux modèles distincts ?

Le Modèle A raisonne sur des chiffres (combien de signalements, depuis quand...) : il est léger et s'entraîne en quelques minutes sur un simple processeur (CPU).

Le Modèle B raisonne sur du langage : il exige des données locales annotées et un modèle de langage adapté à l'Afrique. Son entraînement est plus lourd mais reste maîtrisé (voir §6).

Les mélanger serait une erreur : ils n'utilisent pas les mêmes données ni les mêmes techniques.

1.2 Contrainte fondatrice : léger, rapide, efficace

L'entraînement **ne doit jamais saturer le PC ni le serveur, quelle que soit la taille des données**. Cette contrainte guide tous les choix techniques :

- **En production** : le serveur ne fait que de l'inférence (prédire), opération quasi instantanée qui ne charge pas la machine.
- **Entraînement du Modèle A** : algorithmes légers (arbres de décision boostés), entraînaibles en minutes sur CPU, même avec beaucoup de données.
- **Entraînement du Modèle B** : réalisé hors-ligne (Google Colab / PC), jamais sur le serveur de production ; techniques d'allègement (voir §6.4) pour rester faisable sur une machine modeste.
- **Apprentissage continu** : les mises à jour sont incrémentales et planifiées, jamais un réentraînement complet à chaque requête.

2 Comprendre l'entraînement d'un modèle

Cette section pose les bases pour toute l'équipe. Le cycle est le même pour les deux modèles.

2.1 Le principe : apprendre à partir d'exemples étiquetés

Un modèle apprend en observant de nombreux exemples pour lesquels on connaît déjà la bonne réponse. Cette « bonne réponse » s'appelle l'étiquette (label). Exemple : « ce numéro → frauduleux », « ce message → légitime ». Le modèle ajuste ses paramètres internes jusqu'à prédire correctement l'étiquette, puis on l'utilise sur des cas inconnus.

2.2 Les 7 étapes du cycle de vie

1. **Collecter** des exemples étiquetés (numéros/messages avec leur verdict).
2. **Préparer** : nettoyer les données et les transformer en caractéristiques exploitables (features).
3. **Séparer** en trois paquets : entraînement (~70 %), validation (~15 %), test (~15 %). On n'évalue jamais sur des données déjà vues.
4. **Entraîner** : le modèle apprend à réduire ses erreurs sur le paquet d'entraînement.
5. **Évaluer** sur le paquet de test avec des métriques adaptées à la fraude (§7).
6. **Déployer** : exposer le modèle via l'API d'inférence appelée par le backend.
7. **Boucler** : les nouveaux signalements deviennent de nouvelles données → réentraînement (apprentissage continu).

2.3 Le vocabulaire essentiel

Terme	Signification simple
Feature (caractéristique)	Une information mesurable donnée au modèle (ex. nombre de signalements d'un numéro).
Label (étiquette)	La bonne réponse connue (ex. frauduleux / légitime).
Entraînement	Phase où le modèle apprend à partir des exemples.
Inférence (prédiction)	Phase où le modèle donne une réponse sur un cas nouveau (rapide, en production).
Sur-apprentissage	Le modèle « récite » les exemples mais échoue sur du nouveau : à éviter via la validation.
Démarrage à froid	Au lancement, on a peu de données : on compense par des règles expertes (§5).

3 Stratégie de données (adaptée au Cameroun / Afrique)

La donnée est l'actif le plus précieux du projet. Un bon modèle sur de mauvaises données échoue ; un modèle simple sur de bonnes données locales réussit. Cette section répond à la question centrale : comment collecter les bonnes données ?

3.1 Sources de données, de la plus fiable à la plus bruitée

Source	Description	Fiabilité
Signalements KWISMO	Chaque signalement dans l'app = une donnée étiquetée, gratuite et parfaitement locale.	Très haute
Règles expertes	Verdicts produits par des règles au démarrage (numéro très signalé, préfixe incohérent...).	Haute
Collecte terrain	Dons de SMS/messages d'arnaque par les premiers utilisateurs (avec consentement), captures de vishing.	Haute (locale)
Partenariats	Opérateurs, associations de consommateurs, listes de numéros signalés.	Variable
Jeux publics	Données SMS spam publiques : utiles pour démarrer le NLP mais non camerounaises.	Amorçage seulement
Augmentation	Génération de variantes réalistes d'arnaques locales, relues par un humain.	Complément

3.2 Le défi des langues locales

Les messages frauduleux au Cameroun mélangent français, anglais, *pidgin* et franglais, avec des fautes volontaires et un style non professionnel. Un modèle entraîné uniquement sur de l'anglais standard échouerait. La solution tient en une phrase : **la donnée locale annotée par des humains qui comprennent le contexte est irremplaçable.**

- **Annotation humaine** : les deux data scientists (et des relecteurs locaux) étiquettent les messages collectés. C'est la moitié du travail, autant que la modélisation.
- **Modèle de langue africain** : on s'appuie sur un modèle pré-entraîné sur des langues africaines (famille AfroXLMR/AfriBERTa) qui « comprend » déjà mieux le pidgin que les modèles anglophones standards (voir §6).

3.3 Gouvernance & conformité

- Les numéros de téléphone sont des données personnelles : traitement conforme à la réglementation locale en vigueur.

- Le consentement est recueilli par l'application mobile (hors périmètre de ce document).
- **Minimisation & pseudonymisation** : on ne conserve que le nécessaire ; les identifiants sont pseudonymisés pour l'entraînement quand c'est possible.

3.4 Éthique & biais (à surveiller)

Éviter la discrimination par préfixe ou région

Un modèle pourrait injustement sur-signaler tous les numéros d'un préfixe ou d'une région simplement parce que quelques fraudeurs y sont apparus.

Mesures : surveiller le taux d'alerte par préfixe/région, exiger un faisceau d'indices (pas un seul), et permettre la contestation d'un statut. La réputation se fonde sur le comportement, jamais sur l'origine seule.

4 Modèle A — Scoring de réputation des numéros

C'est le cœur du service au lancement. Il alimente la vérification de numéro, la liste de contacts à insignes et le transfert protégé.

4.1 Caractéristiques (features) utilisées

Caractéristique	Pourquoi elle est utile
Nombre de signalements	Un numéro très signalé est très probablement frauduleux.
Vitesse des signalements	Une montée soudaine indique une campagne de fraude active.
Ancienneté du numéro	Les numéros jetables récents sont plus risqués.
Opérateur & préfixe	Cohérence pays/opérateur/préfixe ; certains schémas sont suspects.
Diversité des signaleurs	Beaucoup d'utilisateurs différents = signal plus fiable qu'un seul.
Historique de vérifications	Fréquence et résultats des vérifications passées.
Statut communautaire	Verdicts antérieurs validés par les administrateurs.

4.2 Algorithme

- **Choix principal** : arbres de décision boostés (gradient boosting, type LightGBM/XGBoost). Excellents sur données tabulaires, très rapides à entraîner sur CPU, robustes et explicables.
- **Repli simple** : régression logistique comme référence de base (baseline), encore plus légère.
- **Sortie** : un score de risque de 0 à 1, converti en statut (sécurisé / à signaler / frauduleux) selon des seuils réglables (§7).

Pourquoi pas un réseau de neurones ici ?

Sur des données tabulaires (des chiffres), les arbres boostés sont souvent aussi précis et beaucoup plus légers/rapides qu'un réseau de neurones. C'est le choix « soft, rapide, efficace » demandé.

4.3 Entraînement

- S'entraîne en quelques minutes sur un CPU ordinaire, même avec un grand volume de données.
- Réentraînement continu léger : planifié (ex. quotidien) ou déclenché par un seuil de nouvelles données, jamais à chaque requête.

- L'explicabilité (importance des caractéristiques) aide à justifier une alerte auprès de l'utilisateur et à détecter les biais.

5 Règles expertes — protéger dès le premier jour

Au lancement, il y a peu de signalements : le modèle manque de données. Les règles expertes protègent les premiers utilisateurs et génèrent des étiquettes qui nourriront le modèle.

- Un numéro signalé au-delà d'un seuil par des utilisateurs distincts est marqué « à signaler » / « frauduleux ».
- Une incohérence pays/opérateur/préfixe déclenche une prudence.
- Ces règles servent aussi de repli quand le modèle est indisponible (robustesse, voir §9).

Transition douce

À mesure que les données s'accumulent, le Modèle A prend le relais des règles, qui restent en filet de sécurité. Les deux coexistent : le modèle décide, les règles garantissent un plancher de protection.

6 Modèle B — Analyse de texte multilingue (NLP)

Ce modèle lit un message (SMS/WhatsApp) et estime s'il s'agit d'une arnaque. Il vise le contexte camerounais/africain avec l'approche avancée demandée, réalisée de la manière la plus simple et légère possible.

6.1 Prétraitement du texte

- Normalisation : minuscules, nettoyage des caractères parasites, gestion des émojis et des répétitions (« gagnéééé »).
- Conservation des indices d'arnaque : montants, numéros courts, mots-clés (« code », « OTP », « gagné », « MTN/Orange »), sans sur-nettoyer.
- Prise en charge du mélange de langues (français/anglais/pidgin) sans forcer une seule langue.

6.2 Modèle de langue africain

- **Base** : un modèle pré-entraîné sur des langues africaines (famille AfroXLMR / AfriBERTa), qui comprend déjà mieux le pidgin et le français que les modèles anglophones.
- **Affinage (fine-tuning)** : on spécialise ce modèle sur les messages KWISMO annotés (arnaque / légitime). C'est l'étape qui l'adapte réellement au terrain.
- **Écosystème** : Hugging Face Transformers (chargement du modèle, entraînement, sauvegarde) ; PyTorch en support. TensorFlow reste possible mais Hugging Face est privilégié pour ce type de modèle.

6.3 Repli simple garanti

Filet de sécurité si la machine ne suit pas

Si l'entraînement du modèle africain s'avère trop lourd pour la machine disponible, un modèle simple TF-IDF + régression logistique sert de repli : très léger, entraînable sur n'importe quel PC, et déjà utile.

La production reste dans tous les cas légère : seule l'inférence tourne sur le serveur, et un petit modèle de texte prédit en une fraction de seconde.

6.4 Entraînement léger — les techniques clés

Pour respecter la contrainte « ne jamais saturer la machine », l'entraînement du Modèle B s'appuie sur des techniques d'allègement, en ligne (Colab) ou sur PC :

Technique	Effet
Google Colab / Kaggle (GPU gratuit)	Déporte l'entraînement lourd hors du serveur de production, gratuitement.
Fine-tuning léger (PEFT / LoRA)	On n'entraîne qu'une petite partie du modèle : bien plus rapide et léger.
Quantification	Réduit la taille du modèle pour l'inférence : rapide et peu gourmand en mémoire.
Distillation (option)	Transfère la connaissance vers un modèle plus petit pour la production.
Petits lots (batches) & early stopping	Limite la mémoire utilisée et arrête l'entraînement dès que ça n'améliore plus.

Résultat

Un entraînement faisable sur Colab gratuit ou un PC modeste, et un modèle final quantifié qui prédit vite sur le serveur sans le saturer.

7 Évaluation & métriques

En fraude, les cas frauduleux sont rares. Un modèle qui dirait « tout est sûr » aurait un fort taux de bonnes réponses mais serait inutile. On utilise donc des métriques adaptées.

Métrique	Question à laquelle elle répond	Enjeu KWISMO
Rappel (recall)	Quelle part des fraudes réelles attrape-t-on ?	Crucial : rater une fraude = utilisateur volé.
Précision	Parmi les alertes, combien sont justes ?	Trop de fausses alertes = les gens ignorent l'app.
Score F1	Équilibre entre rappel et précision.	Mesure de synthèse principale.
Courbe précision/rappel	Comment évolue le compromis selon le seuil.	Aide à choisir les seuils d'alerte.
Matrice de confusion	Où le modèle se trompe (faux positifs/négatifs).	Diagnostic et suivi des biais.

7.1 Seuils d'alerte réglables

- Les seuils qui transforment un score en statut (sécurisé / à signaler / frauduleux) sont réglables sans réentraîner le modèle.
- On privilégie un rappel élevé sur les cas graves (ne pas rater de fraude), tout en surveillant les fausses alertes.

8 Apprentissage continu & gestion des versions

- **Automatique** : conformément aux diagrammes du projet, l'entraînement se déclenche automatiquement à partir des nouvelles données (vérifications, signalements), sans intervention d'administrateur.
- **Incrémental & planifié** : mises à jour légères et périodiques ; jamais un réentraînement complet à chaque requête.
- **Versioning des modèles** : chaque modèle entraîné est versionné (numéro de version, date, performance). On peut revenir à une version précédente en cas de régression.
- **Registre de modèles** : les artefacts de modèles sont stockés dans un emplacement convenu (voir §9.3), avec leurs métriques.

9 Service d'inférence & intégration avec le backend

Le service IA est une API Python séparée (FastAPI). Il **ne se connecte jamais à la base de données** : il reçoit ses données uniquement du backend et lui renvoie ses prédictions.

9.1 API d'inférence

Route	Rôle
POST /predict/number	Reçoit les caractéristiques d'un numéro, renvoie score + statut (Modèle A).
POST /predict/text	Reçoit un message, renvoie la probabilité d'arnaque (Modèle B).
POST /feedback	Reçoit une nouvelle donnée étiquetée (signalement) pour l'apprentissage continu.
GET /health · GET /version	État du service et version du modèle chargé.

9.2 Modes de communication

- **Synchrone** : le backend appelle /predict lors d'une vérification et reçoit une réponse immédiate.
- **Asynchrone** : les signalements sont transmis via /feedback pour nourrir l'apprentissage continu.
- **Robustesse** : si le service IA est momentanément indisponible, le backend applique ses règles expertes de repli — l'utilisateur n'est jamais bloqué.

9.3 Cohabitation dans le dépôt Git (point critique)

Le service IA et le backend vivent dans **le même dépôt Git**, en deux sous-dossiers distincts (kwismo-ai/ et kwismo-backend/). L'exigence : quand l'entraînement se termine, le modèle doit être **immédiatement exploitable** sans manipulation ni incompatibilité. Trois règles le garantissent.

- 8. Contrat d'API partagé** : les formats d'échange (entrée/sortie de /predict) sont définis par des schémas communs, documentés dans les deux Swagger. Aucun des deux services ne devine le format de l'autre.
- 9. Emplacement de modèle convenu** : le modèle entraîné est sauvegardé dans un dossier d'artefacts partagé (monté en volume Docker) ou publié dans un registre. Le service IA y charge la dernière version au démarrage et après chaque réentraînement.
- 10. Orchestration commune** : un docker-compose à la racine du dépôt démarre backend + base + service IA ensemble, avec les variables (URL du service IA, chemin des modèles) déjà câblées. Un seul « docker compose up » lève tout.

Ce que ça garantit

Dès la fin de l'entraînement (sur Colab ou PC), l'artefact de modèle est déposé à l'emplacement convenu ; le service IA le charge et le backend l'utilise via l'API — sans friction ni incompatibilité de format.

Les deux dossiers restent indépendants (équipes séparées) mais parlent le même langage.

10 Architecture & structure des dossiers

10.1 Choix : Cookiecutter Data Science (adapté)

La structure suit la convention **Cookiecutter Data Science**, standard reconnu des projets de science des données, adaptée pour inclure le service d'inférence. Elle sépare clairement les données, l'exploration, le code source et les modèles.

Pourquoi ce choix (justificatifs pour l'équipe)

- **Standard reconnu** : structure comprise par tout data scientist ; onboarding rapide.
- **Séparation données / code / modèles** : on ne mélange pas les données brutes, les notebooks d'exploration et le code de production.
- **Reproductibilité** : chaque étape (préparation, entraînement, évaluation) est un script versionné, rejouable à l'identique.
- **Compatible Colab** : les notebooks d'entraînement s'exécutent aussi bien en local que sur Colab, avec le même code source importé.

10.2 Arborescence normative

```

kwismo-ai/
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_exploration.ipynb
│   ├── 02_train_model_a.ipynb
│   └── 03_train_model_b.ipynb
├── src/
│   ├── config.py
│   ├── data/
│   │   ├── collect.py
│   │   ├── clean.py
│   │   └── features.py
│   ├── models/
│   │   ├── model_a/
│   │   │   ├── train.py
│   │   │   ├── predict.py
│   │   │   └── rules.py
│   │   ├── model_b/
│   │   │   ├── preprocess.py
│   │   │   ├── train.py
│   │   │   ├── predict.py
│   │   │   └── fallback.py
│   ├── evaluation/
│   │   └── metrics.py
│   └── api/
│       ├── main.py
│       ├── schemas.py
│       └── loader.py
├── models/
│   ├── model_a/
│   ├── model_b/
│   └── registry.json
├── tests/
└── .env / .env.example

# Données (jamais versionnées si sensibles)
# Données brutes reçues (via le backend)
# Données nettoyées intermédiaires
# Données prêtes pour l'entraînement
# Exploration & entraînement (local + Colab)
# Scoring numéros
# NLP AfroXLMR
# ★ Code source réutilisable
# Chemins, seuils, hyperparamètres
# Réception des données depuis le backend
# Nettoyage
# Construction des caractéristiques
# Scoring de réputation
# Règles expertes (démarrage à froid)
# NLP multilingue
# Fine-tuning (Colab/PC)
# Repli TF-IDF
# Rappel, précision, F1, matrices
# ★ Service d'inférence (FastAPI)
# Routes /predict, /feedback, /health
# Contrat d'API partagé avec le backend
# Chargement de la dernière version du modèle
# ★ Artefacts entraînés (partagés au backend)
# ex. model_a_v3.pkl
# modèle NLP quantifié
# Versions + métriques (versioning)
# Tests des pipelines et de l'API
# BACKEND_URL, MODEL_DIR...
```

```

└─ requirements.txt      # scikit-learn, lightgbm, transformers...
└─ Dockerfile           # Image du service d'inférence
└─ README.md

```

Le pont avec le backend

Le dossier `models/` contient les artefacts entraînés ; c'est l'emplacement partagé (monté en volume) que le service d'inférence charge et que l'orchestration commune relie au backend. `src/api/schemas.py` porte le contrat d'API partagé : il DOIT rester cohérent avec les schémas côté backend (module `ai_gateway`).

10.3 Stack technique

Usage	Bibliothèque
Manipulation de données	Pandas, NumPy
Modèle A (scoring)	scikit-learn, LightGBM (ou XGBoost)
Modèle B (NLP)	Hugging Face Transformers, PyTorch, PEFT (fine-tuning léger)
Repli NLP	scikit-learn (TF-IDF + régression logistique)
Service d'inférence	FastAPI + Uvicorn
Suivi d'expériences (option)	MLflow ou simple registre JSON
Qualité & tests	pytest, Ruff, Black

11 Organisation des deux data scientists

Les deux data scientists travaillent en parallèle, dans le même dossier, sur des responsabilités complémentaires.

Data 1 — Scoring & pipeline	Data 2 — NLP & annotation
Modèle A : caractéristiques, entraînement LightGBM, seuils.	Modèle B : prétraitement, fine-tuning AfroXLMR, repli TF-IDF.
Règles expertes (démarrage à froid).	Collecte & annotation des messages locaux.
Pipeline de données (collecte, nettoyage, features).	Prétraitement du français/pidgin.
Service d'inférence <code>/predict/number + /feedback</code> .	Service d'inférence <code>/predict/text</code> .
Évaluation & versioning du Modèle A.	Évaluation & surveillance des biais.

Point de coordination

Les deux partagent le contrat d'API (`src/api/schemas.py`) et le dossier `models/`. Une revue croisée régulière garantit la cohérence avec le backend.

12 Livrables attendus

- 11.** Pipeline de données : collecte (via backend), nettoyage, construction des caractéristiques.
- 12.** Modèle A entraîné (scoring de réputation) + règles expertes de démarrage/repli.
- 13.** Modèle B entraîné (NLP AfroXLMR) + repli TF-IDF garanti.
- 14.** Service d'inférence FastAPI (/predict/number, /predict/text, /feedback, /health, /version).
- 15.** Contrat d'API partagé cohérent avec le backend (module ai_gateway).
- 16.** Évaluation documentée (rappel, précision, F1) et seuils réglés.
- 17.** Versioning des modèles (registre) + artefacts déposés dans le dossier partagé.
- 18.** Dockerfile du service + intégration au docker-compose commun.

Fin du cahier des charges — KWISMO Modèle IA